PRANAV PRASAD



+91 8939589630
pranavprasadworks18@gmail.com
https://mystrident.vercel.app
https://github.com/Mystrident
https://www.linkedin.com/in/pranav-prasad-b02510410/

PROFILE

Computer Science and Engineering undergraduate at SASTRA Deemed University, specializing in Cyber Security and Blockchain Technology. Builds full-stack products and systems-oriented software, with practical experience in endpoint telemetry, backend services, database consistency, operating-system integration, and low-level C programming.

Currently learning LLD, HLD to design better software.

EDUCATION

SASTRA Deemed University — B.Tech. in Computer Science and Engineering (Cyber Security and Blockchain Technology) Thanjavur, Tamil Nadu

Relevant coursework: Data Structures and Algorithms, Operating Systems, Computer Networks, Database Management Systems, Cryptography and Cryptanalysis, Linux Programming and System Calls, Object Oriented Programming.

TECHNICAL SKILLS

Languages: C, C++, Java, JavaScript, TypeScript, SQL, Shell Scripting.

Backend & APIs: Node.js, Express.js, REST APIs, Socket.IO, JWT.

Frontend: React.js, Tailwind CSS.

Databases: MongoDB, PostgreSQL, Prisma.

Systems & Platforms: Linux, Windows Services, Named Pipes IPC, macOS launchd, Swift.

Tools & Integrations: Git, Razorpay, Cloudinary, Nodemailer.

EXPERIENCE

Software Engineering Intern | LCode Technologies | June 2026 – July 2026 | Chennai, Tamil Nadu

Worked on a cross-platform endpoint-monitoring system for Windows and macOS that collected antivirus product, version, scan, and license-expiry telemetry and reported it to a centralized backend.

Work delivered

- Developed endpoint telemetry collection using TypeScript and Node.js, including inspection of antivirus-related system information.
- Separated Windows telemetry collection from service lifecycle management: a C#/.NET 8 Windows Service handled persistent background execution, while the endpoint process performed system inspection.

- Implemented Named Pipes IPC between the Windows Service and endpoint process, with tray-based control for user-session interaction.
- Built telemetry ingestion and persistence using Node.js, Express, PostgreSQL, and Prisma.
- Implemented backend checks for overdue scans, license expiry, and missing endpoint heartbeats.
- Implemented a native Swift launchd daemon for macOS, working with platform-specific service management, permissions, and background-process diagnostics.

Engineering challenges

- Cross-platform lifecycle differences: Windows Services and macOS launchd have different service-management and permission models. The implementation required platform-specific execution paths rather than assuming one background-process model would work everywhere.
- Privilege and process separation: persistent service execution and user-session controls have different security and lifecycle requirements. The Windows design separated the service from the endpoint process and used Named Pipes for communication.
- Background-process diagnostics: macOS daemon setup required understanding launchd configuration, permissions, and diagnostics for processes that do not run like ordinary interactive applications.
- Telemetry-to-backend flow: endpoint observations had to be represented, transmitted, persisted, and evaluated centrally so the backend could identify overdue scans, expiry conditions, and missing heartbeats.

Learning outcomes

- Windows service architecture, process lifecycle, and IPC using Named Pipes.
- macOS launchd, native Swift integration, permissions, and background-process troubleshooting.
- Cross-platform systems design and the separation of privileged background work from user-session interaction.
- Backend telemetry ingestion, relational persistence with PostgreSQL/Prisma, and health-condition checks.

Technologies: TypeScript, Node.js, C#, .NET 8, Swift, Express.js, PostgreSQL, Prisma, Named Pipes, launchd.

Developer | VelaCafe — Café Ordering Platform | May 2026 – Present | Live: https://vela-cafe.vercel.app

Designed, developed, and deployed a full-stack café ordering platform to digitize customer ordering and support café operations. The product has recorded 4,000+ sign-ups and approximately ₹6,000 in revenue.

Work delivered

- Built the application using React, Node.js, Express.js, and MongoDB, implementing customer-facing ordering workflows and backend services.
- Designed inventory reservation using atomic conditional updates and MongoDB transactions to reduce overselling risk when multiple customers attempt to order the same stock.
- Implemented time-limited stock reservations and scheduled recovery logic to restore inventory after reservations expire.
- Unified browser payment verification and payment webhooks through an idempotent order-finalization flow.
- Used server-side payment snapshots and database uniqueness constraints to handle duplicate payment callbacks safely.

- Implemented JWT-authenticated Socket.IO rooms for real-time order updates and authorized customers before allowing access to their order channels.

Engineering challenges

- Concurrent inventory updates: multiple customers can attempt to reserve the same item at nearly the same time. Atomic conditional updates and transactional operations help ensure that stock changes are based on the current database state rather than stale application reads.
- Reservation expiry and side effects: TTL-based deletion alone is insufficient when expiration must also restore reserved inventory. A separate scheduled recovery path was implemented to perform the required side effect.
- Duplicate payment finalization: browser verification and payment webhooks can both report the same payment. A unified idempotent finalization flow, backed by server-side payment data and uniqueness constraints, prevents duplicate callbacks from creating repeated finalization effects.
- Real-time channel authorization: Socket.IO rooms must not become an access-control shortcut. Customers are authenticated and authorized before joining their order-specific channels.

Learning outcomes

- Practical use of atomic database operations, transactions, and concurrency control in a deployed application.
- Designing idempotent payment workflows and reasoning about duplicate asynchronous callbacks.
- Understanding why expiry mechanisms that delete records do not automatically perform business side effects.
- Applying authentication and authorization to real-time communication channels.
- Translating operational requirements into backend invariants and recovery logic.

Technologies: React, Node.js, Express.js, MongoDB, Socket.IO, JWT, Razorpay, Cloudinary, Nodemailer.

WSA Internship

Full-Stack Music Player | GitHub: https://github.com/Mystrident/fullstack-music-player

A MERN application built to practise connecting a React interface to backend APIs and persistent data through a complete application workflow.

Work delivered

- Developed frontend and backend components across a full-stack application.
- Worked with React component-based UI development, server-side routing, API integration, and database-backed application state.

Engineering challenges & learning outcomes

- Worked across frontend and backend boundaries, tracing application behaviour through UI components, API requests, server-side routing, and persistence.
- Developed a clearer understanding of separating presentation, API, and persistence responsibilities and debugging issues across application layers.

Technologies: React, Node.js, Express.js, MongoDB.

SELECTED PROJECTS

LSB Steganography in C | GitHub: https://github.com/Mystrident/c_lsb_steganography

Implemented encoding and decoding of secret data in image files using least-significant-bit manipulation, changing selected bits while aiming to preserve the carrier image's visual appearance.

Work delivered

- Implemented encoding and decoding paths that transform payload data into bits and reconstruct it during decoding.
- Used C file streams, byte-level processing, pointers, and bitwise operators to manipulate image data.

Engineering challenges

- Maintaining consistent data ordering between encoding and decoding so that the decoder reconstructs the original payload correctly.
- Handling payload length and byte-level representation so the decoding path can determine how much embedded data to recover.
- Working directly with binary file streams and individual bits rather than treating image contents as ordinary text.

Learning outcomes

- Strengthened understanding of binary representation, bitwise operations, pointers, and low-level file I/O.
- Learned to reason about encoder/decoder symmetry, payload boundaries, and byte-level debugging.

Technologies: C, binary file I/O, pointers, bitwise operations.

MP3 Tag Reader | GitHub: https://github.com/Mystrident/c_mp3_tag_reader

Built a C command-line utility to read and modify MP3 metadata such as title, artist, album, and track information by processing metadata at the binary-file level.

Work delivered

- Implemented metadata reading and modification for supported tag fields.
- Used C structures and file operations to interpret tag data and update metadata without treating the audio file as plain text.

Engineering challenges

- Handling binary offsets and metadata fields accurately when locating and interpreting tag information.
- Maintaining correct file positioning and field handling while modifying metadata in a binary file.
- Avoiding assumptions that an MP3 file can be safely processed as ordinary text; metadata operations require structured binary parsing.

Learning outcomes

- Improved understanding of binary file formats, C structures, pointers, file positioning, and low-level I/O.
- Developed more careful debugging habits for offset-sensitive file operations.

Technologies: C, structures, pointers, binary file handling.